

Reviewing a Task and Creating `review.csv`

0. What this document is

A trainer-facing walkthrough for creating `review.csv`: what each of the 14 review areas checks and how to score it, the exact file format and status rules `review.csv` must follow, the fix-and-verify protocol for anything you correct, and where to actually generate the file.

 [review.csv review form \(Google Apps Script\)](#)

1. The five audit layers, at a glance

Layer	What it means	Human role
Layer 1	Task-package validity and semantic consistency: whether <code>task.toml</code> , <code>instruction.md</code> , <code>solution/</code> , <code>tests/</code> , inputs, outputs, and environment jointly define one coherent, realistic, gradable task.	Human review required — assess consistency, clarity, realism, leakage, and whether the golden solution and verifier match the task as written.
Layer 2	Checked-in vendor QC evidence: difficulty rollouts, a non-oracle passing trajectory for solvability, and repeated grading of identical deliverables for stability.	No manual review required — audited programmatically against <code>task_folder/evaluations/</code> . (<i>Programmatic checks should exist for this.</i>)
Layer 3	Oracle validation: Harbor runs the oracle solution once per task in E2B and Modal, requiring an exact reward of 1.0 in both.	No manual review required — runtime completion, artifact materialization, verifier execution, and strict rewards are checked programmatically.
Layer 4	Model-and-harness runtime evaluation: saved rollouts	Human review required — confirm attempts are

Layer	What it means	Human role
	(trajectories, artifacts, rewards, environment behavior, connector interactions) from the configured model/agent lanes.	reasonable and grounded, tools/files/connectors work, outputs are usable, and failures aren't infrastructure artifacts.
Layer 5	Qualitative analysis of every Layer 4 trial: rollout reasonableness, verifier fairness, environment/connector behavior, artifact integrity, failure attribution.	Human review required — validate automated explanations, investigate flags, detect reward hacking or false grading, and decide whether the task needs changes.

Practical takeaway: You are the human reviewer for **Layers 1, 4, and 5, plus Layer 2 · Difficulty** — that's everything you actually score.

Separately, exactly **three** checks are run entirely by the Turing team, not by you:

- Layer 2 · Solvability
- Layer 2 · Stability
- Layer 3 · Oracle Mode

2. Who does what

Area	Who runs it	In review.csv
Layer 2 · Difficulty (the 4 GLM runs)	You — assess whether the task is difficult enough	PASS / FIXED_AND_VERIFIED , based on your review
Layer 2 · Solvability	Turing team	N/A
Layer 2 · Stability	Turing team	N/A
Layer 3 · Oracle Mode	Turing team	N/A

Area	Who runs it	In review.csv
Everything else (Layer 1, Layer 4, Layer 5, Cross-trial · Calibration)	You	PASS / FIXED_AND_VERIFIED, based on your review

⚠ **Rule:** for the three Turing-run checks (Layer 2 · Solvability, Layer 2 · Stability, Layer 3 · Oracle Mode), just put **N/A in every cell of the row** — **status**, **review_notes**, **change_made**, and **what_to_record**. No explanation needed; do **not** try to produce or restate their evidence yourself.

3. The **review.csv** file — format requirements

review.csv is the auditable record of what was checked, what failed, what changed, and whether the fix actually worked. Attach it to the Harbor task package at every review checkpoint.

Required columns, in this exact order:

None

review_check, **status**, **review_notes**, **change_made**, **what_to_record**

Column	Required content
review_check	The applicable layer or check from the Human Review Guide (see §5 below for the full list).
status	PASS, FIXED_AND_VERIFIED, or N/A.
review_notes	What was inspected and the conclusion — the running history of your review.
change_made	The exact correction made. Blank only when no change was needed.
what_to_record	The evidence tied to the "What to record" column for that check in the Human Review Guide.

Submission eligibility rule: a task is ready for submission only when `review.csv` contains **exactly one row per applicable layer/area**, and **every row is `PASS` or `FIXED_AND_VERIFIED`**. Any missing row, unresolved failure, blocker, or unverified correction blocks submission.

Quality bar for notes: notes like *"looks good," "fixed," or "reran"* are **not sufficient**. Write enough that someone else could confirm what you actually reviewed without re-doing the work.

4. Status definitions

- **PASS** — the check passed and supporting evidence is provided. Nothing was changed.
- **FIXED_AND_VERIFIED** — a real problem was found, the package was changed, **and** the change was rerun through the relevant Harbor path with the original issue confirmed gone.
- **N/A** — the check doesn't apply, with a brief explanation (e.g., the connector row on a task with no connectors).

 **Making a change is not enough.** A failure can only be closed once the check that originally exposed it has been repeated and the post-fix evidence confirms the result. There is no **FAIL** status — a row you can't honestly resolve means the task isn't finished yet, not something to record as failing.

5. Fix-and-verification protocol

For every failed check, in order:

1. Record the original failure and its evidence.
2. Record the exact change made.
3. Rerun the relevant check — via the relevant automated QC check or a Harbor re-run.
4. Recheck any other backend, verifier, artifact, or rollout affected by the change.
5. Add the post-fix evidence and reference the original finding.
6. Mark the row **FIXED_AND_VERIFIED** **only** when the original issue no longer reproduces.

If the rerun fails or exposes a new issue, keep the item open and add the new result to the notes — don't overwrite or remove the original finding.

6. The 14 review areas — full detail

Use these as your working checklist. Each one below becomes exactly one row in `review.csv` (extra rows are fine if you find something the standard 14 don't cover — held to the same bar).

Layer 1 · Package consistency

- **Review question:** Do `task.toml`, `instruction.md`, every file under `solution/`, and every file under `tests/` describe the same executable and gradable task?
- **Evidence to inspect:** Read all four sources. Compare required outputs, paths, formats, inputs, services, timeouts, scoring rules, and expected values.
- **Pass criteria:** The instruction asks for what the golden solution produces and the verifier grades; `task.toml` supplies everything both require.
- **Red flags:** Undeclared required files or fields; contradictory paths or formats; verifier-only requirements; stale manifests; missing dependencies.
- **What to record:** Cite the conflicting files and exact requirement, explain the impact, and propose one canonical fix.

Layer 1 · Clarity and scope

- **Review question:** Is the task well defined for an agent that only sees the declared instruction and environment?
- **Evidence to inspect:** Instruction, input inventory, connector descriptions, output requirements, and verifier expectations.
- **Pass criteria:** A capable agent can identify the goal, available evidence, required deliverables, and completion criteria without guessing hidden rules.
- **Red flags:** Ambiguous terms; missing units or date ranges; unstated assumptions; multiple plausible answers; hidden precision or formatting demands.
- **What to record:** Describe the ambiguity, plausible interpretations, and the smallest instruction or verifier change that resolves it.

Layer 1 · Realism and leakage

- **Review question:** Does this resemble useful real work without exposing the answer or relying on artificial traps?
- **Evidence to inspect:** Instruction, packaged inputs, seeded connector data, filenames, environment image, solution visibility, and test fixtures.
- **Pass criteria:** The scenario, inputs, tools, and outputs are plausible; necessary complexity follows from the work; golden facts are not agent-visible.
- **Red flags:** Contrived identifiers or traps; impossible access assumptions; answer-bearing filenames; golden files mounted into the workspace; toy output with no practical value.

- **What to record:** State whether the task is realistic, identify leakage or contrivance, and suggest a more natural framing where needed.

Layer 2 · Difficulty (the 4 GLM runs)

- **Review question:** Is the task difficult for GLM-5.2?
- **Pass criteria:** GLM-5.2 passes ≤ 2 of 4 runs.
- **Red flags:** GLM-5.2 passes ≥ 3 of 4 runs (too easy).
- **What to record:** How many of the 4 GLM-5.2 runs passed.

Layer 2 · Solvability (*Turing team runs this — mark N/A in review.csv*)

- **Review question:** Is this task solvable by a frontier model given only the task + environment?
- **Evidence to inspect:** If no passing trajectory exists from multiple frontier models across multiple repeats, determine whether that's genuine difficulty (good) or a sign the task is ill-defined / verifiers are inconsistent.
- **Pass criteria:** A human or AI can find a trajectory reaching a perfect reward of 1.0 using only the task + environment, **and** a human agrees the verifiers are the right checks for the task.
- **Red flags:** No pass trajectory across multiple frontier models and multiple repeats.
- **What to record:** A single successful run among the repeats attempted (e.g., "1 of 6 passes").
- **In review.csv:** put **N/A** in every cell of this row (status, review_notes, change_made, what_to_record). No explanation needed.

Layer 2 · Stability (*Turing team runs this — mark N/A in review.csv*)

- **Review question:** Does repeated grading of the same rollout produce the same output? Ideally run on the solvability pass trajectory.
- **Pass criteria:** Same reward across 3 repeats of the verifier run.
- **What to record:** Confirmation of the same reward across 3 repeats.
- **In review.csv:** put **N/A** in every cell of this row (status, review_notes, change_made, what_to_record). No explanation needed.

Layer 3 · Oracle Mode (*Turing team runs this — mark N/A in review.csv*)

- **Review question:** Does the Harbor CLI run the oracle solution and produce reward 1.0, in both E2B and Modal sandbox backends?
- **Evidence to inspect:** Note that oracle $\neq$ solvability — oracle only confirms consistency between the oracle trajectory and the verifier, not that the task itself is solvable by a model.
- **Pass criteria:** Oracle-mode agent/verifier results confirm 1.0 reward in both backends.
- **What to record:** The oracle-mode results confirming 1.0 reward for all tasks.

- In **review.csv**: put **N/A** in every cell of this row (status, review_notes, change_made, what_to_record). No explanation needed.

Layer 4 · Environment and files

- **Review question:** Can the agent reliably use the sandbox, dependencies, inputs, workspace, and required output paths?
- **Evidence to inspect:** Trajectory setup steps, filesystem operations, dependency installation, logs, produced artifacts, exception details, and E2B environment metadata.
- **Pass criteria:** Inputs are present and readable; required tools run; writes persist; the verifier sees the intended artifacts; failures are task-related, not infrastructure-related.
- **Red flags:** Missing files; incompatible architecture; dependency-build failures; permission errors; outputs written to the wrong workspace; artifacts disappearing before verification.
- **What to record:** Name the failed interaction and classify it — task packaging, sandbox, dependency, harness, or agent error.

Layer 4 · Connectors, MCPs, and CLIs (*can be N/A*)

- **Review question:** Are intended connectors discoverable, usable, and sufficient for the task?
- **Evidence to inspect:** Declared MCP/CLI metadata, tool discovery, calls and responses, authentication behavior, pagination, state changes, and verifier-side database checks.
- **Pass criteria:** The agent can discover the interface, read all necessary records, perform allowed writes, handle pagination, and confirm resulting state without hidden credentials.
- **Red flags:** Service never starts; missing auth; wrong endpoint; incomplete pagination; unusable schemas; silent tool errors; verifier expecting state the connector can't create.
- **What to record:** Connector name, failed or successful operations, affected trial, and whether the issue is data, service, auth, interface, or agent usage. Mark **N/A** for non-connector tasks.

Layer 4 · Deliverables and artifact quality

- **Review question:** Are the requested outputs complete, realistic, and usable in their declared formats?
- **Evidence to inspect:** Instruction output list, workspace artifacts, golden files, JSON structure, and any PDF, DOCX, spreadsheet, HTML, Markdown, or text deliverables.
- **Pass criteria:** Every required artifact exists, opens/parses, contains substantive requested content, and uses a format appropriate to the work.
- **Red flags:** Placeholder or malformed files; JSON-only grading ignoring required rich output; polished-looking artifacts lacking substance; verifier checking the wrong or stale path.
- **What to record:** List missing or weak artifacts, and distinguish content quality, format validity, path, and materialization issues.

Layer 5 · Verifier coverage and fairness (*most important*)

- **Review question:** Does the verifier measure the task requirements accurately and proportionately?
- **Evidence to inspect:** *tests/*, per-check results, weights, reward aggregation, LLM rubric text and explanations, golden artifacts, and agent outputs. A reward of 1.0 is *evidence*, not *proof* of task quality — a fractional or zero reward may reflect an agent error, task defect, verifier defect, or infra problem.
- **Pass criteria:** A verifier is justified and checks a non-ambiguous expectation of the task. Verifiers cover all query expectations, accept valid equivalent solutions, reject substantive errors, and match the instruction.
- **Red flags:**
 - Query requirements not reflected in the verifier suite
 - Brittle verifiers that assign a deterministic expectation to an ambiguous requirement
 - Duplicate checks / excessive overlap
 - Regex too restrictive when multiple equally valid answers exist
 - Deterministic/regex checks used where a model could accidentally pass, or be penalized for a semantically-correct-but-differently-worded answer
 - Instruction doesn't specify results must be in the final message (not just output files), and the judge penalizes the omission
 - Judge misses parts of the trajectory and gives an incorrect verdict
 - Feasibility issues (e.g., agent lacks access to a required Slack channel)
 - Stated deterministic judges (weighted into the final score) that don't actually have a working verifier behind them (e.g., SQL checks with no verifier)
- **What to record:** Each verifier must have a justification referencing the part of the query that outlines the expectation it's checking. Identify ambiguous, defective, or missing checks; quote the requirement in your own words; propose a concrete verifier fix.

Layer 5 · LLM judge consistency

- **Review question:** Are qualitative judgments grounded in the actual artifact and stable enough to trust?
- **Evidence to inspect:** Vendor stability repeats, judge model/config, artifacts, and verifier stdout.
- **Pass criteria:** Explanations cite observable evidence, don't invent missing content, and repeated judgments don't flip materially on identical deliverables.
- **Red flags:** Factually wrong rationale; judge overlooking present content; rubric demanding unstated detail; inconsistent repeats; invalid model/provider configuration.
- **What to record:** The disputed rubric item, artifact evidence, repeat behavior, and recommended rubric or judge configuration change.

Layer 5 · Reward hacking and exploitability

- **Review question:** Could an agent earn a high reward without genuinely completing the intended task?
- **Evidence to inspect:** Trajectory, final artifacts, verifier implementation, golden data exposure, state assertions, path handling, and reward aggregation.
- **Pass criteria:** Reward tracks genuine completion; shortcuts, spoofed files, hardcoded answers, prompt injection, and stale state cannot satisfy the verifier.
- **Red flags:** Agent reads golden data; writes directly to verifier state; mimics expected strings without evidence; exploits unchecked fields; skips required side effects yet passes.
- **What to record:** Describe the exploit path, affected checks and reward, severity, and an acceptance test that closes it.

Cross-trial · Calibration

- **Review question:** Across four trials, does the task have a healthy difficulty and behavior profile?
- **Evidence to inspect:** Strict passes across all four trials, fractional rewards, exceptions, model/harness differences, automatic flags, and all Layer 5 findings.
- **Pass criteria:** Results are neither trivially universal nor universally blocked; model differences are plausible; fractional scores and exceptions have understandable causes.
- **Red flags:**
 - Zero passes because infrastructure is broken
 - Passes across the board with shallow behavior
 - Systematic harness-only failure
 - Fractional rewards driven by one defective rubric item
 - A SOTA model fails a criterion that weaker models pass
 - All models fail a particular criterion with the same answer (suggests a broken verifier, not genuine difficulty)
- **What to record:** Summarize difficulty, suspicious cross-trial patterns, and highest-priority problems, if any.

7. Dummy **review.csv** example

A complete, filled-out example — all fourteen rows, correct header, correct quoting, and the three Turing-run rows marked **N/A** as described above — is provided as a standalone dummy **review.csv** so you can see a valid full file rather than a couple of isolated rows.

 [Dummy review.csv example](#)

Note on the N/A rows in that example: the Connectors row uses N/A with an explanation, because the general rule is that N/A means "doesn't apply to this task." The three Turing-run rows are a deliberate exception — they *do* apply, but you don't produce the evidence — so for those three, just put N/A in every cell and move on; no explanation needed.

Quoting note: any cell containing a comma must be wrapped in double quotes, and a literal double quote inside a cell is doubled (""). Writing the file in a spreadsheet and exporting as CSV handles both for you automatically — hand-editing raw CSV text is where "could not be read as CSV" errors usually come from.

8. Delivery and acceptance checklist

A task package is ready for acceptance only when:

- ☐ Every applicable review check has a recorded status.
- ☐ Every failure has a documented cause and disposition.
- ☐ Every claimed fix has been rerun and verified.
- ☐ No original failure was overwritten or removed (new findings get added, not swapped in).
- ☐ The task package runs through the required Harbor paths without undocumented workarounds.
- ☐ `review.csv` has exactly one row per applicable layer/area, and every row is **PASS** or **FIXED_AND_VERIFIED**.

A task with unresolved failures, unverified fixes, missing references, or incomplete notes is **not** ready for acceptance.

9. Where to actually create `review.csv`

Don't hand-write the CSV. Use the review form (Google Apps Script) — it enforces the exact header and the fourteen areas so the file it produces can't be rejected on shape:

1. Open the form and sign in with your Turing Google account.
2. Paste the Drive link to the **extracted task folder** (the folder, not the zip).
3. Fill in *Status*, *Review notes*, *Changes made*, and *What to record* for all fourteen (or more) questions.
4. **Submit and upload to Google** writes `review.csv` straight into that Drive folder, or **Download review CSV** gives you the file to place yourself.

5. To edit later, paste the same folder link and use **Fetch review CSV** to reload what you already wrote.

 [Review form link](#)

 [Human Review Guide \(source spreadsheet, full detail + layer table\)](#)

Then confirm the file is **inside the ZIP you upload** — attaching it on the task page unblocks Submit, but only the archive that's actually uploaded reaches the client.

Sources

This guide merges and reorganizes two source documents. They are the authority if anything here disagrees with them:

- [Turing Review Guide — Human Review Guide \(Google Sheet\)](#) — the 14 review areas, review questions, evidence to inspect, pass criteria, red flags, and what to record for each (source for §1, §2, §6).
- [ComputerBench Quality Control Instructions for Turing](#) — the client's spec for the `review.csv` deliverable: required columns, status definitions, and the fix-and-verification protocol (source for §3, §4, §5).